Data and Tool Assessment for Case Study 2 - Leveraging Contextual Data for Use in Sampling Frames and Estimation

In developing Case Study 2, the project team evaluated numerous data sources and data processing tools for each step of data preparation and processing, with a focus on open-source tools, for tasks ranging from geocoding to image classification. The team then selected a preferred tool and implemented that tool in the case study. The team also evaluated several data sources for satellite and street view imagery.

This document summarizes the processing steps, provides context about each type of tool and resource, and summarizes tool and source evaluations.

1. Tools

Address to Geocode Conversion and Within-Radius Coordinate Sampling

The team explored open-source tools in R that can be used to identify the radius around a given address and sample coordinates within that radius. The process converts a list of addresses into geocoded coordinates, generates circles with a specified radius around those coordinates, and samples coordinates within those circles. The process involves three primary steps.

1. Preparation

Prepare a file with a list of addresses. To be compatible with most standard address geocode APIs, the file will contain street address (number, name, and direction), city, state, and ZIP code. For example:

6936 Nancy Ave, Portage, IN 46368

1905 New Haven Ct SE, Smyrna, GA 30080

6389 State Hwy 298, Benton, AR 72019

The file should be in a file format that can be read by the selected tool (such as a file that can be read by R for the R package censusxy,[1] a wrapper for communication with the Census Geocoder API).

2. Convert to Geocodes

Send the cleaned address list to the geocoder (such as to the Census Geocoder API using the censusxy package). This results in a dataframe with addresses and their corresponding longitude and latitude. Specifying a "full" output option in the censusxy package also gives the nearest TIGER line and identifies on which side of the line the address is located.

For this step the team evaluated two potential tools, the Census Geocoder and Google Map Services. A comparison of the two is provided in Table 1.

[1] Other packages include:
R: https://github.com/SigmaMonstR/census-geocoder
Python: https://pypi.org/project/census-geocoder/
Python: https://github.com/ClayGendron/usgeocoder
Java: https://github.com/taurenk/Java-Geocode r

Table 1: Comparison of Geocoding Tools

	Census Geocoder	Google Maps Services Geocoding API
Description	Converts address to geocode: latitude/longitude interpolated from address endpoints associated with TIGER/Line segment, or corresponding geographies (e.g., census block) Can manually go to the API website to upload the csv file with addresses or use programming packages to interface with the API. R package censusxy automatically processes the JSON returned from the API and gives back a dataframe with longitude and latitude with "simple" output and more details including match quality and TIGER database line ID (portion of the street between two intersections) with "full" output	Converts address into latitude/longitude, using multiple sources, returned in JSON or XML format Can set up a workflow in Python using their Python client
Strengths	Automatically batch processes for cases over 1,000 Can immediately view the coordinates using the mapview/leaflet/ggplot2 package if you specify the return object class as "sf" (simple features, R dataframe for spatial data). Free, no account set up Easy to use	Relies on diverse proprietary sources of data Google collects, so could survive Census Bureau deciding to change or shut down their API A JavaScript demo is available on their documentation site: https://developers.google.com/maps/documentation/geocoding/overview
Limitations	Census Bureau may change how the API works. Due to technical problems resulting from API changes, the R package has been removed from CRAN and has to be manually installed from github https://github.com/chris-prener/censusxy. Core functionality worked as of April 2026.	Need set up a GCP account on "essential tier" (credit card info required) Limited to 10,000 free requests per month quota Maximum 90 day free trial

3. Generate Radius and Sampling Coordinates

Once the geocoding is complete, use a geographic analysis tool to create a circle (generally treated as a polygon, i.e. it does not necessarily have to be a circle) around the coordinates.

Note that in some systems the coordinates have to be projected into a planar Coordinate Reference System (CRS). Otherwise, the model assumes that the coordinates are on a flat surface which causes the sf package to generate a jittery circle.

The team evaluated three potential tools. A comparison is provided in Table 2.

Table 2: Comparison of Radius and Sampling Coordinates Tools

	Freemap Tools	R package sf[2]	R package geosphere
Description	Develop radius Can view on the map within the website or manually generate a KML file to export from the website Must have a CSV file with columns for longitude and latitude and a specified radius	General GIS spatial manipulation Supports drawing circles with a specified radius around given coordinates Supports sampling within specified radius	Geographic distance and coordinate calculations Allows generating random coordinates within a specified radius
Strengths	KML file works with Google maps (traditional format) Simple to use for a some cases	Straightforward and streamlined process all within R studio Supports immediately proceeding to sampling coordinates within the specified radius	Simple method for generating random points around a center location Accounts for the Earth's curvature Does not require CRS transformation

[2] R package sf https://r-spatial.github.io/sf/
Simple Features for R https://r-spatial.github.io/sf/

	Freemap Tools	R package sf[2]	R package geosphere
Limitations	Must copy and paste the CSV contents into the web browser instead of uploading the file and then download the KML file to use it in R The z axis (sea level) must be dropped to work with mapview One wrong row with typos or extra columns will cause the application to fail	Needs projected coordinate system CRS 3857 instead of directly using lat/long values due to Earth curvature, otherwise produces a jittery circle	None identified

Detecting Nearest Intersection of Geocode Coordinates

The team tested procedures used to identify the nearest intersection for sampled coordinate points. Our assessment focused on freely available open tools in Python and Google Colab. The workflow focuses on comparing multiple methods for nearby intersection detection, including an OSMnx-based approach, an Overpass-based approach, and a rule-based approach adapted from an existing “Detect Nearby Crossroad” notebook.

1. Preparation

Begin with a file containing latitude and longitude coordinates (usually in CSV format). These points may represent sampled locations in the study area, such as Detroit, or out-of-state test locations used for comparison. For example:

42.345796, -82.948209
42.312669, -83.264903
42.294744, -83.042840

The file should be saved in a location accessible from Google Colab, such as Google Drive, so that it can be read and processed during the workflow.

After the coordinate file is uploaded or linked, it is read into Python as a pandas dataframe. The team ran all three methods in Google Colab to ensure a consistent environment.

The main packages used in the workflow include:

- pandas for tabular data input and output
- requests for querying the Overpass API

- osmnx for downloading and analyzing OpenStreetMap road networks
- folium for visualizing query points and detected intersections on an interactive map
- distance functions such as haversine or geodesic calculations for measuring straight-line distance between coordinates

This setup allows multiple intersection-detection strategies to be applied to the same set of sampled points.

2. Processing

In processing, the team assessed three methods.

OSMnx-based Nearest Intersection Method

The first method uses OSMnx to download a drivable road network and represent it as a graph, where nodes are connection points and edges are road segments. Candidate intersections are identified based on the number of distinct roads connected to each node.

For each sampled point, the method searches for the nearest valid candidate intersection in the road network. This approach is efficient for large-area and batch processing, but in this project it was not always the most accurate when checked against manual inspection.

Overpass-based Nearest Intersection Method

The second method uses the Overpass API to directly query OpenStreetMap data around each sampled point. Roads and nodes are retrieved within a series of search radii, and a node is treated as a candidate intersection if it belongs to at least two distinct road identifiers after filtering out unhelpful road types.

This method offers more direct control over the raw OpenStreetMap elements and makes it easier to inspect how an intersection is identified. In this project, it often produced the most accurate results because it preserved local road topology more directly than the graph-based approach. Although it better matched the intended interpretation of the nearest intersection in many cases, it depends on live API queries, may require retry logic, and can become slow when applied to larger datasets.

Detect Nearby Crossroad Rule-based Method

The third method adapts an existing Detect Nearby Crossroad notebook for batch processing in Google Colab. It focuses mainly on named main roads, such as motorway, trunk, primary, secondary, and tertiary roads, while also checking residential roads to help filter overlapping names. Road names are standardized so that minor naming differences, such as “Street” versus “St” or “East” versus “E,” are treated consistently. Candidate crossroads are then identified as nodes shared by at least two distinct roads, and the closest valid crossroad is selected.

Because this method emphasizes major named roads, it often returns intersections on larger roads rather than on smaller neighborhood streets. It is therefore useful as a comparison method, but not as the main production method in this workflow.

Progressive Search Radius

To improve coverage, the methods can use a progressive search-radius strategy. Instead of relying on a single radius, the code checks a sequence such as 300m, 600m, 1200m, 2500m, and 5000m. If no valid intersection is found at a smaller radius, the search expands to the next level. This reduces failed cases while still favoring nearby intersections.

3. Generate Output

After all points are processed, the results were saved as a CSV file. The output usually includes:

- query latitude and longitude
- detection status
- search radius used
- detected intersection latitude and longitude
- straight-line distance from the query point to the intersection
- associated road identifiers or road names
- node identifier, when available

These outputs make it possible to compare the three methods side by side and review successful and unsuccessful cases. Visual inspection provided an important quality check for whether the detected intersection was reasonable. This step highlighted cases where a method may have selected a *major* road intersection instead of the *closest* intersection.

The three methods do not always return the same result because they reflect different definitions of a nearest intersection:

- OSMnx emphasizes overall road-network structure
- Overpass emphasizes raw local topology from queried OpenStreetMap elements
- Detect Nearby emphasizes named main roads and rule-based standardization

In this project, the Overpass-based method better preserves local road topology and more closely matches the intended definition of the nearest intersection. However, for larger datasets, OSMnx is often the more practical choice because it is more scalable for batch processing, while the Overpass-based method can become slow when many points must be queried individually.

The team evaluated three potential tools. A comparison is provided in Table 3.

Table 3: Comparison of Nearest Intersection Tools

	Overpass API	OSMnx	GeoNames
Description	Queries the OpenStreetMap Overpass API to retrieve nearby road segments around each coordinate using a progressive search radius (300m–5,000m). The returned roadways are decomposed into nodes, and nodes belonging to at least two distinct road segments are treated as candidate intersections. After filtering pedestrian paths and driveway-type roads, the nearest candidate node is selected as the road-network anchor point.	Downloads the local road network graph from OpenStreetMap once and finds the nearest network node for each incident location. Records node coordinates as the standardized road anchor point.	Uses the GeoNames nearest-intersection web service to retrieve the closest named street intersection and its coordinates for each location.
Strengths	Direct access to detailed road geometry Performs well in structured grid street networks Flexible radius search Free and open data source	Road network is downloaded once, enabling efficient batch processing Avoids repeated API calls	Returns real named intersections Simple API usage No graph processing required
Limitations	Nearest node is not always a real intersection Processing can be slow because each point requires a separate Overpass API query	Requires downloading large network area Results depend on chosen network boundary	Limited coverage in some regions Less spatial precision than OSM-based methods

Boundary Processing

The team tested procedures for transforming block group boundaries to latitude/longitude coordinates, calculating bounding box coordinates. Our assessment focused on open-source tools.

The OpenStreetMap approach transforms block group boundaries to latitude/longitude coordinates (EPSG:4326), calculates bounding box coordinates, downloads OSM raster tiles

covering the geographic extent, and transforms tiles back to match the block group's original projection. The block group boundary polygon is overlaid on the base map (with the boundary displayed), producing a composite image that is saved as a PNG file named using the block group GEOID, at an adjustable resolution. For example, a resolution of 600 dpi was found to provide good image quality suitable for subsequent analysis and coding, while maintaining reasonable processing time. Image resolution is an important determinant of computation time and should therefore be carefully considered during the design and implementation of the workflow.

Of the tools tested for this step of the workflow, OpenStreetMap offered advantages including no API key requirement, registration, or usage limits. OpenStreetMap also automatically includes highlighted block group boundary overlays, which are essential for identifying geographic units. OpenStreetMap operates entirely within R, which helps maintain workflow consistency, and it provides sufficient street-level detail for reference maps without added complexity. Drawbacks include slower processing when searching for images for certain census block groups, and OpenStreetMap data quality varies across different locations. For example, even at the same image resolution, some geographic areas appear clear while others are less detailed.

Alternatives include Leaflet, which generates interactive web maps but introduces additional complexity due to HTML rendering and the need for screenshot capture when producing static images. ESRI ArcGIS Image, provides higher-resolution outputs (up to 4096×4096 pixels), but returns only base maps without built-in block group overlays, resulting in additional processing steps.

The team evaluated three potential tools. A comparison is provided in Table 4.

Table 4: Comparison of Boundary Processing Tools

	OpenStreetMap (OSM)	Leaflet	ESRI ArcGIS Image
Description	Input boundary shapefiles for geographic units of interest Transforms tract boundary to latitude/longitude coordinates (EPSG:4326) required by OSM	We input boundary shapefiles for geographic units of interest (OR input census tract GEOID and let it finds corresponding boundary in shapefile) Transforms tract boundary to WGS84 coordinates (EPSG:4326) required by Leaflet	We input boundary shapefiles for geographic units of interest (OR input census tract GEOID and let it finds corresponding boundary in shapefile) Transforms tract boundary to WGS84 coordinates (EPSG:4326) required by ESRI API

	Calculates bounding box coordinates for the tract area Downloads OpenStreetMap raster tiles from OSM servers covering that geographic extent Transforms downloaded OSM tiles back to match tract's original projection system Overlays tract boundary polygon on transformed base map (highlighted in blue) Renders composite image combining OSM base layer with tract boundary Saves as PNG file	Creates interactive Leaflet web map with OpenStreetMap tile layer as base Overlays tract boundary polygon on the map (highlighted in blue) Saves interactive map as temporary HTML file Capture screenshot of rendered HTML map Converts screenshot to PNG image file named with GEOID	Calculates bounding box coordinates for the tract area Constructs API request URL with bbox, image size, DPI, and format parameters Sends HTTP GET request to ESRI ArcGIS REST API server ESRI server generates map image on-demand for specified geographic extent Downloads rendered PNG image from ESRI server Saves image file named with GEOID
Strengths	Free Everything can be done within R, using their package	Free Everything can be done within R, using their package Works well if using static image is the goal	Free No API keys, registration, or usage limits encountered Adjustable resolution up to 4096x4096 pixels Everything can be done in R, no manual downloading necessary
Limitations	Requesting identification of census tracts can be a slow process, but speed improves with shapefile as input Data quality seems to vary by location	Requesting identification of census tracts can be a slow process, but speed improves with shapefile as input Data quality seems to vary by location	No tract boundary built-in overlay, returns only base map image (a big con; but can add overlay to the images) Image-only output, cannot add labels, legends, or annotations

Building Detection and Density Estimation From Satellite Images

This project develops an image-based approach to estimate building counts and building density from satellite imagery, with the goal of generating scalable and spatially detailed indicators of the built environment. Traditional data sources, such as survey-based measures, are often limited in spatial resolution, infrequently updated, or costly to obtain. As a result, there is a growing need for alternative methods that can provide consistent and fine-grained information across regions.

To address this, the project leverages high-resolution NAIP aerial imagery in combination with Microsoft Building Footprints. The footprint data provide a reference for building locations, while the imagery captures visual information about the physical environment. By integrating these sources, the pipeline is able to learn how buildings appear in images and automatically extract building-related features.

Rather than relying on pre-aggregated statistics, this approach directly analyzes imagery to identify buildings and derive measures such as building counts and density. This enables a more flexible and scalable framework that can be extended to new regions and used to support downstream statistical analysis, including small-area estimation.

Data and Approach

The analysis is based on two complementary data sources: high-resolution NAIP aerial imagery and Microsoft Building Footprints. The imagery provides detailed visual information about the landscape, while the footprint data offer precise geographic representations of building locations. Together, these sources enable the construction of labeled data for model development.

To prepare the data for modeling, the study area is divided into smaller geographic units (tiles). For each tile, an image is extracted from the NAIP data, and the corresponding building footprints are converted into a binary mask that distinguishes building areas from the background. These image–mask pairs serve as the foundation for training the model to recognize building structures.

Due to the large scale of the full Michigan dataset, it is not computationally feasible to process all available data within the current environment. Therefore, the analysis focuses on a subset of the region (Ann Arbor), which provides a manageable yet sufficiently diverse sample. This subset includes both dense and sparse areas, allowing the model to learn a range of building patterns while maintaining efficiency. After data cleaning and quality control, the model is trained on 8,240 curated tiles, which provide sufficient variation for reliable model learning.

In addition to data construction, several quality control steps were necessary to ensure reliable training. Low-quality or blurry imagery was removed, and tiles with little or no building information were filtered out to reduce extreme class imbalance. These steps were critical to prevent the model from learning trivial patterns (e.g., predicting only background) and to ensure that it captures meaningful building features.

Modeling and Evaluation Strategy

The modeling approach is designed to first identify building locations in images and then derive building-level measures from those predictions. Instead of directly estimating building counts from raw imagery, the pipeline focuses on detecting building regions within each tile. This step is essential, as counting buildings without first identifying their spatial boundaries is unreliable.

Using paired image and mask data, the model learns to recognize visual patterns associated with buildings across different environments. Once building regions are identified, building counts are obtained by detecting distinct structures within each tile. These counts can then be aggregated and used to compute building density.

At the current stage, model evaluation is conducted by comparing predicted building counts against reference counts derived from the Microsoft Building Footprint data. This provides a direct and interpretable benchmark for assessing how accurately the model identifies buildings.

Importantly, this evaluation is performed prior to any comparison with ACS-based measures. This separation ensures that the model is validated against an independent reference and prevents unintended information leakage from downstream data sources. Establishing this intermediate validation step is critical for maintaining the integrity of subsequent analyses.

In addition to count-based evaluation, performance is also assessed in terms of how well building regions are detected. Together, these evaluation criteria ensure that the model is both visually accurate and quantitatively reliable.

The current baseline model achieves an Intersection over Union (IoU) of approximately 0.60, indicating that the model is able to capture building structures with reasonable accuracy.

Aggregation and Output

Tile-level predictions are aggregated to larger geographic units (e.g., block groups) to compute building density, defined as the number of detected buildings per unit area. This provides a standardized measure that enables comparison across regions.

Key Challenges

A substantial portion of the project timeline was devoted to data preparation and problem formulation rather than model training itself. Identifying appropriate labeled data was particularly time-consuming, as the task required constructing image–mask pairs by combining

aerial imagery with Microsoft Building Footprint data. In addition, aligning these two data sources introduced further complexity due to differences in format and spatial structure.

Data quality issues also needed to be addressed, including the presence of blurry or low-quality imagery, which required filtering to ensure reliable training data. Furthermore, the overall dataset size was extremely large, making it computationally infeasible to process all available data within the current environment. This led to the adoption of a subsampling strategy, focusing on a representative subset of the region.

Another key realization during the project was that building counts cannot be reliably estimated directly from images. Instead, it is necessary to first identify building regions through segmentation and then derive counts from the resulting predictions. This required a shift in the modeling approach and contributed to the development time.

At the current stage, model evaluation is conducted by comparing predicted building counts against counts derived from the Microsoft Building Footprint data. This step is intentionally performed prior to any comparison with ACS-based measures, in order to ensure that the model is evaluated against an independent reference. This separation prevents unintended information leakage and maintains the integrity of subsequent analyses.

These challenges reflect the importance of careful data construction and validation in ensuring that the resulting measures are both meaningful and reliable.

The team evaluated five potential tools. A comparison is provided in Table 5.

Table 5: Comparison of Building Detection Tools

	U-NET	SAM1	YOLOv8	detectron2
Description	Performs fully supervised image segmentation, mapping satellite images directly to building masks	Performs prompt-based segmentation using a pretrained foundation model to generate masks for objects in an image	Fast object detection (and optional segmentation via YOLOv8-seg) for counting/locating objects	Research-grade object detection plus instance/panoptic segmentation for detailed object boundaries
Strengths	High accuracy for building segmentation Well-suited for pixel-level prediction tasks Straightforward training pipeline	Strong generalization with pretrained weights Works with limited labeled data Flexible for different segmentation tasks	Very fast training and inference Easy setup Good for large-scale automation and counting tasks	High accuracy Pixel-level masks enable precise building footprints, area, and structural patterns
Limitations	Requires labeled training data May overfit without proper regularization	Strong generalization with pretrained weights Works with limited labeled data Flexible for different segmentation tasks	Less accurate for complex shapes Segmentation quality is more limited	Heavier models Slower training/inference More complex configuration

Object Annotation for Street-Level Images

To support the annotation of street-level images, we reviewed a range of image annotation tools spanning both open-source and commercial options. The objective was to select a platform suitable for research workflows that require multiple annotation types, support for collaboration, and the ability to operate locally. The comparison focused on several dimensions, including whether the tool runs fully local/offline, supported task types (bounding boxes, polygons, keypoints, and classification), collaboration features, export formats, data storage, learning curve and setup complexity, and licensing model.

Open-source tools evaluated included CVAT (Computer Vision Annotation Tool), LabelImg, LabelMe, Label Studio, Make Sense, VGG Image Annotator, and VoTT. Most open-source tools allow fully local deployment and store both images and annotations on the local filesystem, which is advantageous for research reproducibility and data control. However, many lightweight tools such as LabelImg, LabelMe, Make Sense, and VGG Image Annotator primarily support individual annotation workflows without built-in collaboration. These tools typically offer limited automation features and are best suited for small datasets or single-user projects. Some tools, including LabelImg and VoTT, also restrict annotation to bounding boxes, reducing flexibility for more complex coding tasks. While these lightweight tools are easy to install and require minimal setup, their limitations in collaboration and task diversity make them less suitable for larger research efforts.

Among open-source tools, CVAT and Label Studio provide more advanced functionality. Both support multiple task types, including bounding boxes, polygons, masks, and keypoints, and both allow multiple-user collaboration. They also support standard export formats such as COCO and YOLO, which are essential for training computer vision models. CVAT offers stronger built-in collaboration workflows, more mature support for large datasets, and broader format compatibility. The main trade-off is that CVAT requires a more complex setup, typically involving Docker and local server configuration, leading to a steeper learning curve. Despite this, CVAT provides a more scalable solution for annotation-intensive research.

Commercial tools reviewed included Roboflow, Labelbox, SuperAnnotate, Scale AI, Supervisely, RectLabel, and MATLAB Image Labeler. These platforms generally provide stronger collaboration features and often incorporate automation hooks such as model-assisted labeling and pre-labeling. Many commercial tools also support large datasets and offer streamlined interfaces that reduce onboarding time. However, commercial platforms rely on cloud-based infrastructure for data storage and collaboration, which may introduce limitations related to data control, storage quotas, or licensing costs. Some tools, such as Supervisely, offer local deployment options, but full functionality is often restricted to enterprise editions.

For our analysis we selected CVAT as the primary annotation tool for annotating street-level images. CVAT runs fully locally, supports a wide range of annotation task types, enables user collaboration, provides compatibility with common formats such as COCO and YOLO, and can scale to larger datasets. These features align with the needs of labeling visible items from street-level images and preparing data for downstream computer vision modeling.

The team evaluated 14 potential tools. A comparison is provided in Table 6.

Table 6: Comparison of Key Strengths and Limitations in Object Annotation Tools

	Strengths	Limitations
CVAT (docker)	Rich functionalities in terms of task type and data format	
Labellmg (Python)		Only able to code items using bbox Not able to collaborate by assigning different users No longer being developed
Labelme (pip)		Not able to collaborate by assigning different users
Label Studio (pip/docker)	Rich functionalities with short learning curve, compared with CVAT	
Make Sense (browser)		Not able to collaborate by assigning different users
VGG Image Annotator (HTML file)	Very lightweight	Not able to collaborate by assigning different users
VoTT (Microsoft)		Only able to code items using bbox Not able to collaborate by assigning different users No longer being developed
Roboflow*	Supports many annotation formats Provides cloud collaboration and local inference options	Requires cloud usage for annotation Free tier has usage and storage limits

	Strengths	Limitations
Labelbox*		Paid service, expensive for large datasets
SuperAnnotate*	Offers auto-track and AI-assisted magic tools to automated labeling	Local deployment requires enterprise license
Scale AI*		No manual labeling UI for users
Supervisely*	Supports 3D annotations AI-powered labeling	Full local deployment requires Enterprise Edition
RectLabel (macOS)*	Lightweight	macOS only No collaboration features.
ImageLabeler (Matlab)*		Requires MATLAB license No collaboration features

*Commercial product, not open-source.

Large Language Models (LLMs)

This project used currently available large language models (LLMs) to evaluate large quantities of street-level images and generate single numerical scores as output. We ran pilot experiments using eight different multimodal LLMs (OpenAI: gpt-4o, gpt-5.1, gpt-5.2; Google: Gemini 2.0 Flash, Gemini 2.5 Pro, Gemini 3 Flash; Anthropics: Claude Opus 4.6, Claude Sonnet 4.5). Through this evaluation we identified 1) preparatory steps needed, 2) end-to-end workflow, 3) cost considerations, and 4) key considerations for implementation.

1. Preparation

Three preparatory steps are required before beginning an LLM-based image evaluation task.

1. A documentation text file should include each prompt as a formal measurement question. Each prompt should indicate the construct being measured with an intended scoring scale. It is recommended to include an instruction such as “Only provide a single numerical score” to constrain output format.
2. A documentation CSV or JSON file should include the source of each image. At the minimum, one column should contain a unique identifier for each image, and another column should contain a stable HTTPS URL that does not require authentication. It is recommended to define image resolution in advance as it affects both measurement and token usage.

3. An API configuration documentation should include the API keys, provider, model name, temperature, top_p, and maximum token settings. API keys should be stored securely (e.g., environment variables). Model providers periodically update systems, so documenting configurations ensures future replication and review.

2. End-to-End Workflow

The workflow begins by reading image URLs from the documentation file and pairing each image with the designated prompts. For each image-question pair, the system sends an individual API call to the selected multimodal model, and submits both the text prompt with the image URL in a single call. Each pair is evaluated multiple independent times to capture variability. All returned outputs with metadata are recorded in a structured dataset and saved as a CSV file for analysis.

3. Approximate the Cost

API pricing for multimodal LLMs is token-based. A token is a unit of text processed by the model. In English text, a token is 3-4 characters on average. Providers bill separately for input tokens (everything sent to the model) and output tokens (everything generated by the model).

Total cost = input tokens x input rate + output tokens x output rate

For image-based tasks, input tokens include two components: textual tokens and image tokens. When an image is sent, it is internally converted into tokens. The number of image tokens depends on image resolution and API detail setting. For most current providers, high-resolution images produce more processing patches, and each patch contributes a fixed token amount. Therefore, image resolution is the primary cost driver.

Using OpenAI's gpt-5.2 under batch pricing as an example:

Input tokens: $0.875 per 1 million tokens

Output tokens: $7.00 per 1 million tokens

Suppose 3,000 images are processed. Each image is evaluated under three prompts, and each image-prompt pair is run five independent times:

3,000 x 3 x 5 = 45,000 API requests

Assume each image is 1024 x 1024 pixels. By OpenAI's documentation, approximate image token usage is:

High detail mode: ~630 image tokens

Low detail mode: ~75 image tokens

Assume 40 prompt tokens and 2 output tokens per request. Under high detail mode:

Input tokens per request: 630 + 40 = 670

Total input tokens: 670 x 45,000 = 30.15 million

Total output tokens: 2 x 45,000 = 90,000

Estimated cost:

Input: 30.15 x $0.875 = $26.38

Output: 0.09M x $7.00 = $0.63

Total = $27.01

Processing 3,000 images under this configuration would cost approximately $25-$30 in total. Using low detail mode would largely reduce the input tokens and lower total cost.

4. Key Considerations

Several technical considerations emerged during the experimentation.

Lack of Replicability: Sending the same prompt and image to a provider's public web interface and through the provider's API may produce different outputs. This can arise for several reasons. The web interface may apply hidden system prompts, safety layers, formatting adjustments, or internal parameter defaults. All production runs should be conducted through the API using explicitly defined parameters. Model name, model version, and any other configurable parameters should be fixed. Testing in web interfaces should not be treated as representative of a certain model's behavior.

Variable Stability: Model parameters influence output stability. Parameters are adjustable settings that control how the model generates outputs. The most relevant parameter in this context is temperature, which controls the degree of randomness in token selection. The model internally processes the image before producing the output. Higher temperature settings increase variability across repeated runs, while lower temperature settings produce more stable outputs. Setting of the parameter influences both dispersion of the outputs.

Quality Control and Documentation: Additional notable matters include the rationale for repeat evaluation, image detail settings, and missingness handling. Repeat evaluation is used to quantify within-model variability. Image detail settings affect both token consumption and visual interpretation, and therefore should be explicitly justified. Missingness protocols should document how invalid image URLs or download failures are identified in the dataset. Clear documentation of these can support the transparency of the workflow.

The team evaluated eight potential tools. A comparison is provided in Table 7.

Table 7: Comparison of Large Language Models for Image Processing

		Standard Price per 1M Tokens		Batch Price per 1M Tokens		
Model Type	**Model Name**	**Input**	**Output**	**Input**	**Output**	**Notes**
OpenAI	gpt-4o	$2.50	$10.00	$1.25	$5.00	Requires tailored prompting strategies, otherwise does not generate effective output
	gpt-5.1	$1.25	$10.00	$0.63	$5.00	Requires tailored prompting strategies, otherwise does not generate effective output
	gpt-5.2	$1.75	$14.00	$0.88	$7.00	Best performing OpenAI model Follows prompts well
Google	Gemini 2.0 Flash	$0.10	$0.40	$0.05	$0.20	Follows prompts better than Gemini 3 Flash
	Gemini 2.5 Pro	Varies $1.25-$2.50	Varies $10.00-$15.00	Varies $0.625-$1.25	Varies $5.00-$7.50	Follows prompts better than Gemini 3 Flash
	Gemini 3 Flash	$0.50	$3.00	$0.25	$1.50	Significantly slower than the other models Variations across same-pair repetitions observed
Anthropic	Claude Opus 4.6	$5	$25	$2.50	$12.50	Most expensive of all models evaluated
	Claude Sonnet 4.5	$3	$15	$1.50	$7.50	Most expensive of all models evaluated

2. Data Sources

Satellite Image Collections

For satellite/aerial Imagery acquisition in the geographic unit of interest, we assessed two potential sources: ESRI World Imagery (API accessible via R), and Mapbox Satellite.

The ESRI World Imagery workflow converts block group boundaries to WGS84 coordinates and sends HTTP requests to the ESRI World Imagery API using bounding box coordinates. Satellite

images are downloaded in PNG format and imported into R as raster objects. Using ggplot2, block group polygons are then layered over the satellite basemap with transparent fills and visible borders, producing composite images. Each image is saved as a PNG file named by the block group GEOID. This approach also allows efficient batch processing, generating one image per block group.

Advantages of ESRI World Imagery include being open source and free to use, with no API keys or registration required and no usage limits. It offers adjustable resolution that enables high-quality imagery and can be implemented directly in R without relying on external servers or web-based platforms.

Limitations include the lack of a built-in block group boundary overlay, which must be added as an additional step during image acquisition. It also has variable satellite imagery quality and differences in image recency across locations.

The alternative, Mapbox Satellite, provides slightly higher-quality imagery and includes a free tier of up to 50,000 requests per month. However, it requires API key registration and credit card or billing information for all users. While this request limit may be sufficient for many use cases, it introduces the risk of exceeding usage quotas and incurring unexpected charges, which requires ongoing monitoring. It also adds administrative complexity and budget uncertainty, making it less suitable for workflows that prioritize simplicity, reproducibility, and cost predictability.

A detailed comparison is provided in Table 8.

Table 8: Comparison of Imagery Sources

	Esri World Imagery	Mapbox Satellite
Description	Transforms tract boundary to WGS84 coordinates (EPSG:4326) Sends HTTP request to ESRI World Imagery API with bounding box coordinates Downloads satellite/aerial photo (PNG) from ESRI servers Reads satellite image into R as raster Uses ggplot2 to layer tract polygon (transparent fill + border) over satellite base Saves composite image as PNG file named with tract GEOID Batch processes multiple tracts, one image per tract	Transforms tract boundary to WGS84 coordinates (EPSG:4326) Constructs Mapbox Static API URL with bounding box and access token Sends HTTP GET request to Mapbox satellite-v9 style endpoint Mapbox server returns high-resolution satellite imagery as JPEG Saves JPEG temporarily and reads into R as raster Uses ggplot2 to layer tract polygon (transparent fill + border) over satellite base Saves composite image as PNG file named with tract GEOID Batch processes multiple tracts one image per tract
Strengths	Free, no API keys or registration Adjustable resolution	High-quality satellite imagery from Mapbox/commercial providers Free tier: 50,000 requests/month Adjustable resolution options available Clear visual representation with highlighted tract boundary (if added by user)
Limitations	No built in tract boundary overlay, but can be introduced as an additional step Satellite imagery age varies by location	Requires API key registration Requires credit card and billing details. While the free tier may suffice, exceeding usage limits can incur charges Satellite imagery age varies by location

Large-Scale Street-Level Image Collections

To support large-scale street-level image collection, the team reviewed various imagery platforms and evaluated each based on geographic coverage, API accessibility, download workflow, and metadata availability. The testing strategy involved selecting 50 random locations within the city of Detroit and 10 random locations across the United States. For each location, we used the geocoded coordinates as the query point to retrieve images from each platform and assess coverage and functionality.

The platforms we reviewed are classified into two categories: commercial platforms and crowdsourced platforms. Commercial platforms include Google Street View, Apple Look Around, Bing Streetside, Yandex(RUS), Baidu (CN), and Kakao Road View(KR). Yandex provides coverage primarily within Russia, Baidu covers mainland China, and Kakao Road View focuses on South Korea. These three provide little coverage of the U.S. and therefore will not be discussed further in this report. Crowdsourced platforms include Mapillary, KartaView, Panoramax, and Mapilio.

Among commercial platforms, Google Street View provides the widest coverage in the United States. It was the only platform able to return imagery for all 60 selected locations in this evaluation. Google Street View offers coverage across urban, suburban, and many rural areas and provides useful metadata, including image ID, latitude/longitude, and capture date. Google's official API operates under usage-based billing and typically returns static perspective images rather than full panoramas. We used the Streetlevel Python library, which enables retrieval of imagery from commercial platforms without requiring API keys. The library sends HTTP requests to the same backend web endpoints used by the platforms' own map viewers, receives structured JSON responses, and translates those responses into Python objects and downloadable images. Using this approach, we were able to extract panoramas and metadata from Google Street View, Apple Look Around, and Bing Streetside within a unified Python environment.

Apple Look Around and Bing Streetside do not currently offer publicly accessible APIs designed for bulk collection, so we relied on the Streetlevel library for retrieval. Apple Look Around provides rich imagery in metropolitan areas, but its coverage is sparse in rural locations. Bing Streetside offers moderate U.S. coverage and can similarly be accessed via Streetlevel. However, its coverage density is lower, and Microsoft's shift toward Azure Maps introduces uncertainty regarding long-term accessibility.

All crowdsourced imagery platforms have lower coverage density than Google Streetview. Mapillary stands out because it offers dense coverage in primarily urban areas, including strong coverage in Detroit. Mapillary provides a fully accessible API that supports bulk download. KartaView is another crowdsourced platform that supports bounding box queries through its API. However, coverage is generally sparse outside certain major cities, and imagery is limited to

single-angle perspective views. Panoramax provides strong coverage in France and most parts in Western Europe but has limited relevance for the United States due to sparse coverage outside Europe.

A detailed comparison is provided in Table 9.

Table 9: Comparison of Street-Level Imagery Sources

	Strengths	Limitations	Camera Type	Find by Map Tiles or Bounding Box	Find by a Point	Picture Format	Metadata Availability
Google Street View*	Widest coverage in the US	Requires usage-based billing	Perspective, Panoramic	Yes	Yes	JPG	Image ID Latitude / Longitude Capture date Heading Elevation
Apple Look Around*	Moderate coverage	For each location, only returns 6 single-facing images of the panorama Sparse coverage in rural areas View only, no public API	Perspective	Yes	No	HEIC	Capture date Heading Elevation
Bing Streetside*	Moderate coverage	API no longer available	Panoramic	Yes	Yes	JPG	Image ID Latitude / Longitude Capture date Heading Elevation
Mapillary	Very dense coverage in Detroit	Sparse coverage outside Detroit	Perspective, Panoramic	Yes	No	JPG	Image ID Latitude / Longitude Capture date Heading Elevation

	Strengths	Limitations	Camera Type	Find by Map Tiles or Bounding Box	Find by a Point	Picture Format	Metadata Availability
KartaView		Sparse coverage in rural areas Only perspective (single facing angle) images available	Perspective	Yes	No	JPG	Image ID Latitude / Longitude Capture date Heading
Panoramax	Very dense coverage in France and western Europe	Sparse coverage outside France and western Europe	Panoramic	Yes	No	JPG	Image ID Latitude / Longitude Capture date Heading
Mapilio		Sparse coverage in general No documented API availability	Perspective	No	No	N/A	N/A
Yandex*		Only Russia	N/A	N/A	N/A	N/A	N/A
Baidu*		Only mainland China	N/A	N/A	N/A	N/A	N/A
Kakao Road View*		Only Korea	N/A	N/A	N/A	N/A	N/A

*Commercial product, rather than crowdsourced.